# Python 視窗遊戲設計 (Pygame)

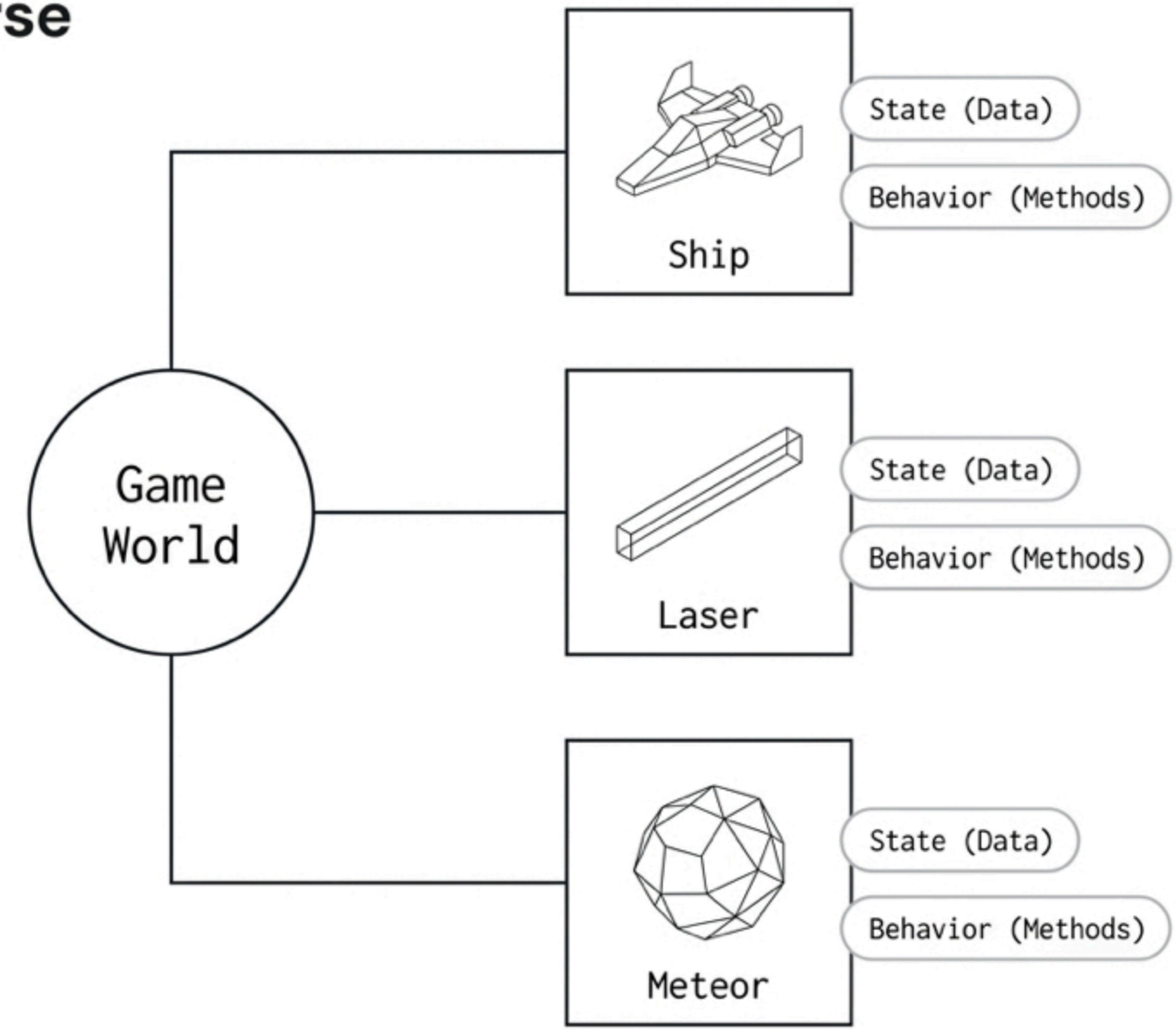
## 第十一章：2D 遊戲引擎、精靈系統與物理碰撞實作

講師：Python 程式設計教學團隊

# The Object-Oriented Universe

In game development, everything is a living object.

- Players, enemies, bullets, and obstacles are independent entities.
- We encapsulate data and logic into distinct, self-governing classes updating in real-time.



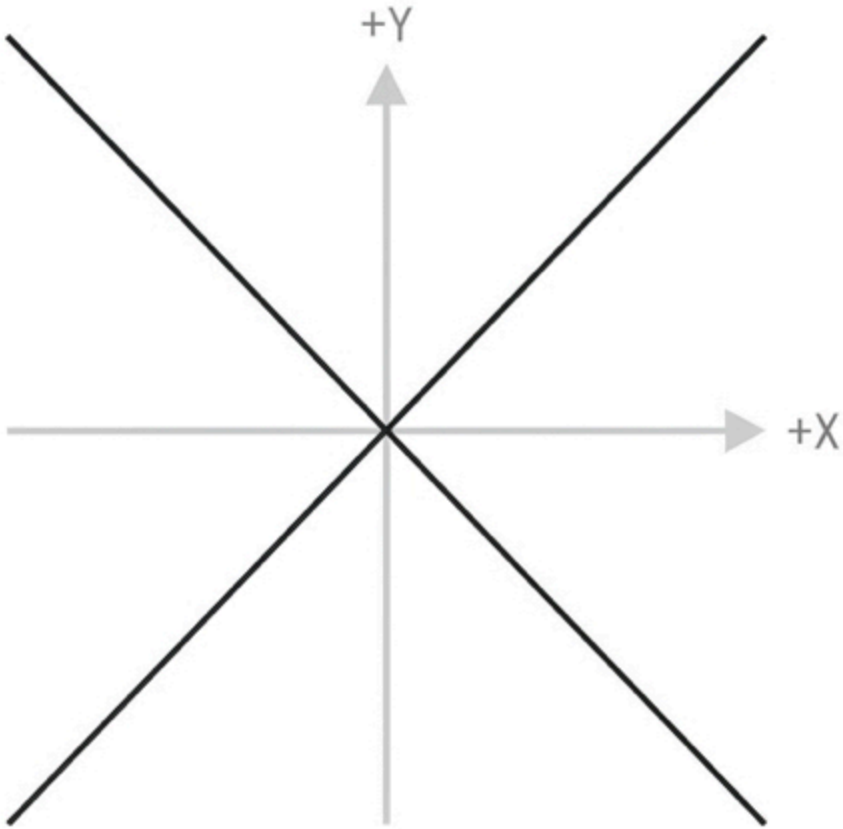

Gemini Notebook

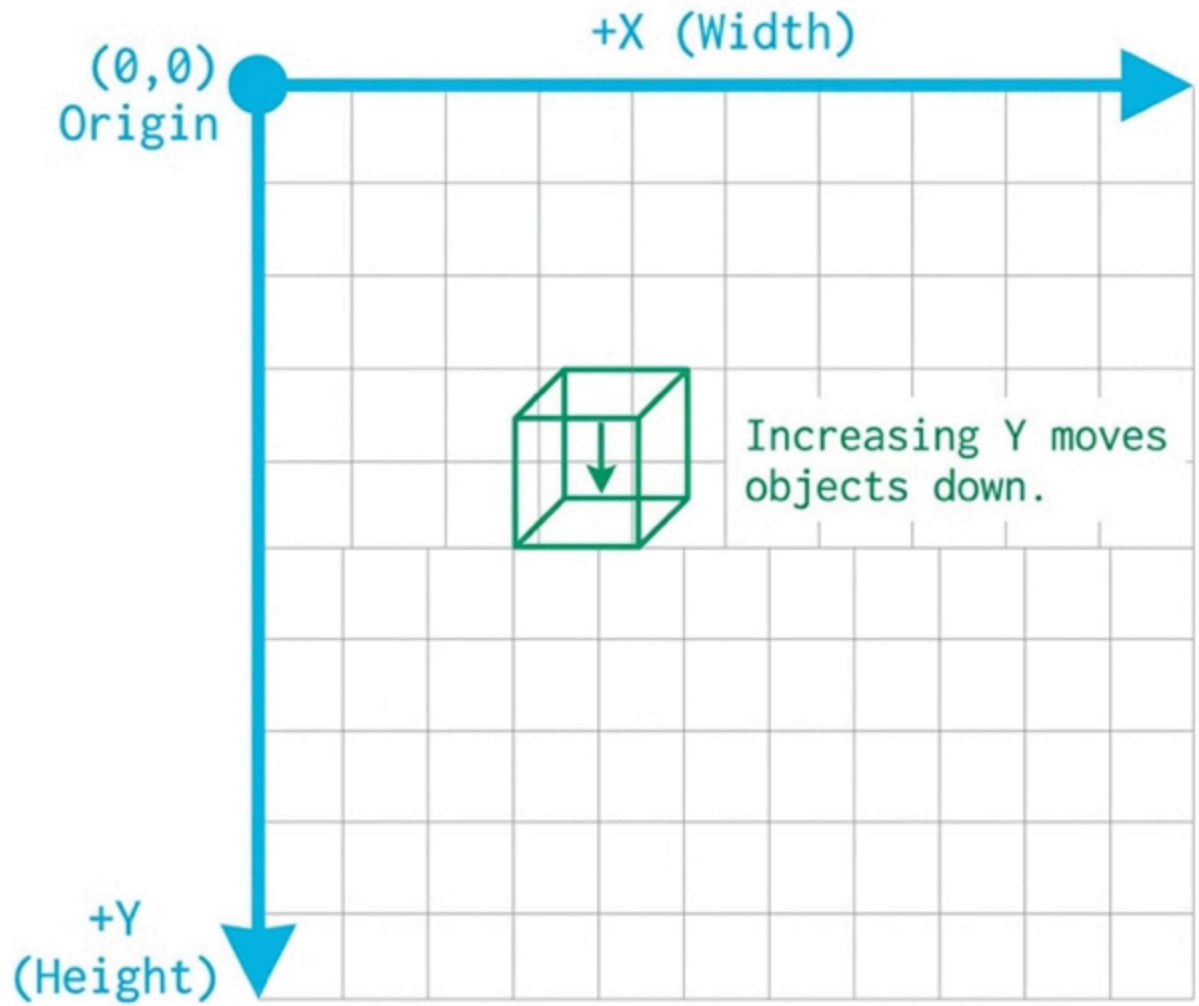
# 11.1 Pygame 與視窗遊戲基礎

- **2D 遊戲開發框架 (Pygame)**：
  - 整合圖形渲染、硬體加速、音訊控制與輸入裝置監聽。
- **螢幕座標系統**：
  - 原點 $(0, 0)$ 位於螢幕**左上角**。
  - 向右為 $+X$，向下為 $+Y$。
- **經典遊戲迴圈 (Game Loop)**：
  - 每秒數十次高速循環：**事件讀取** $\rightarrow$ **狀態更新** $\rightarrow$ **畫面渲染**。

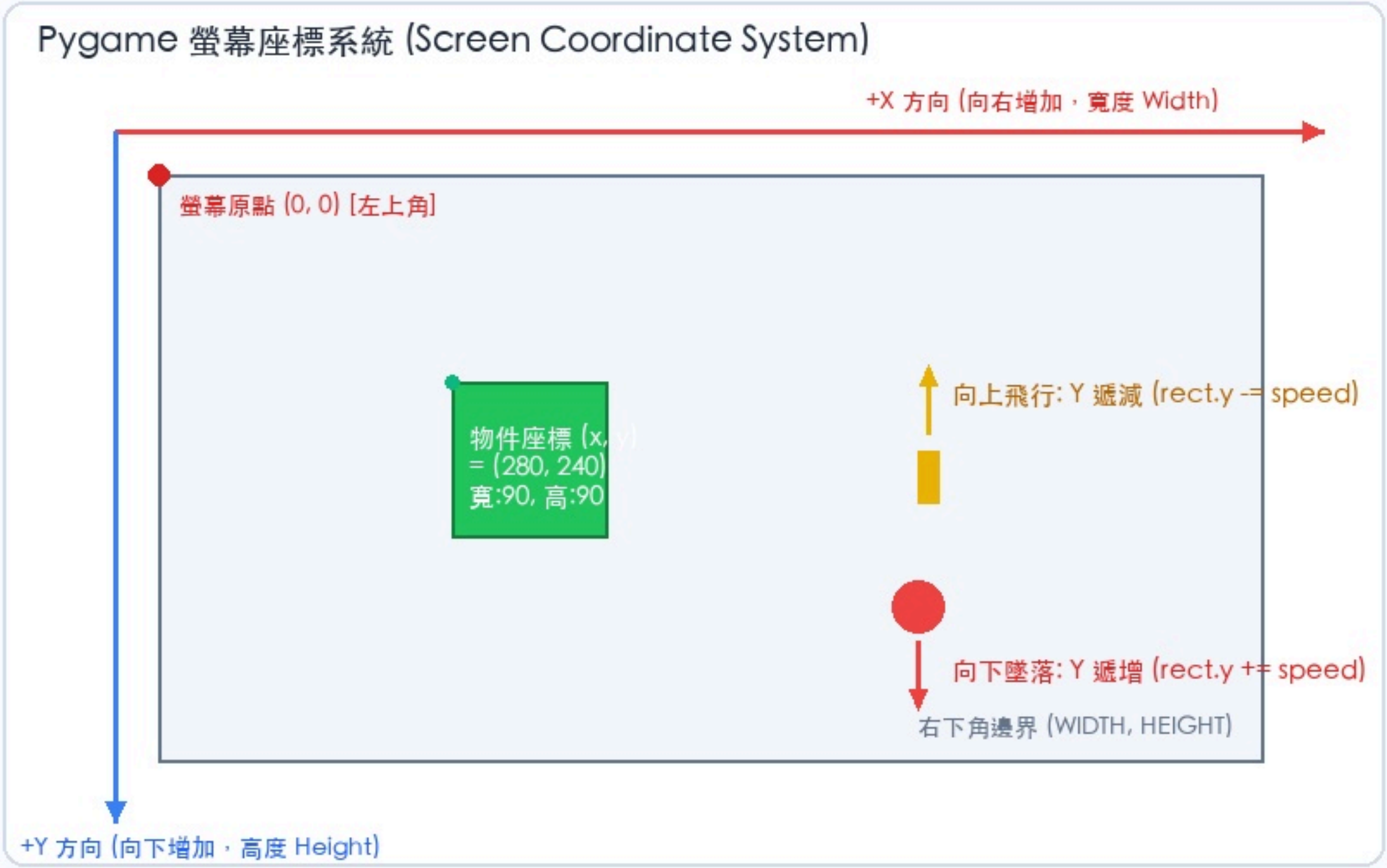
# The Canvas: Redefining Space

Unlike traditional math, computer displays draw from the top-left downward.




Gemini Notebook

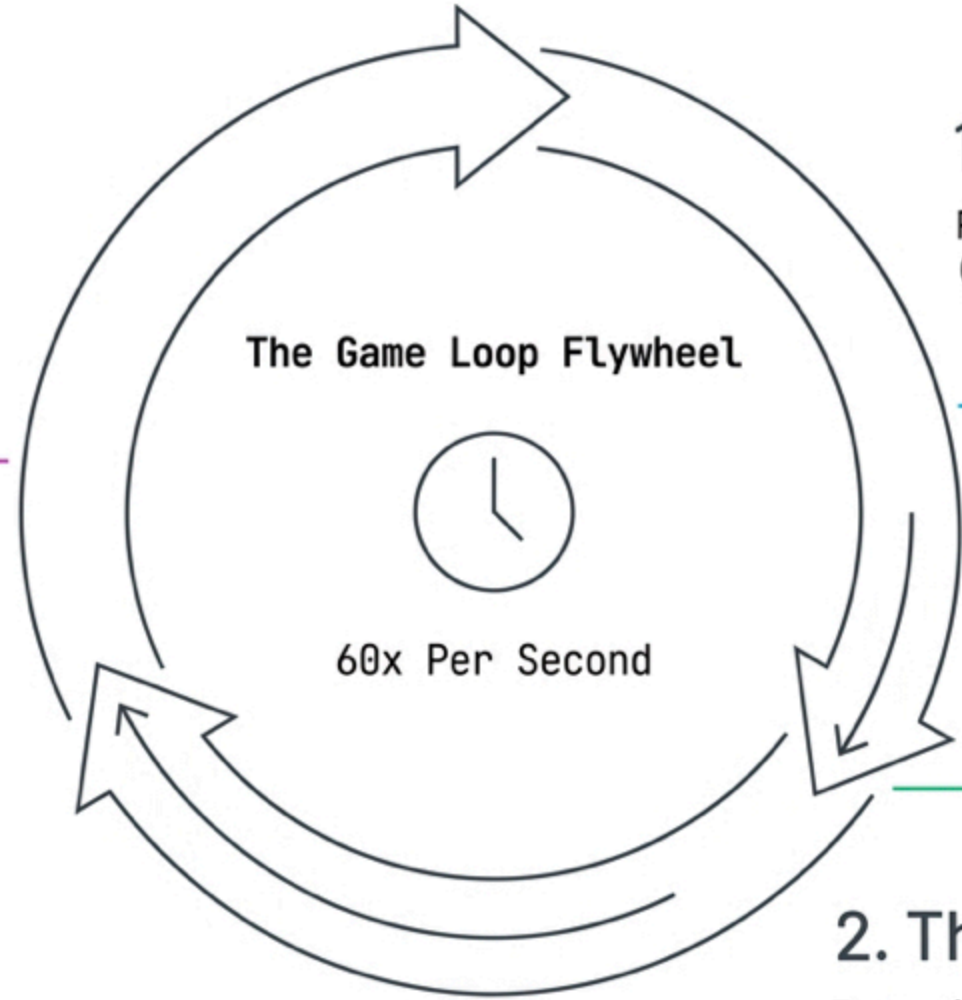

Pygame 螢幕座標系統 (Screen Coordinate System)
+X 方向 (向右增加，寬度 Width)
螢幕原點 (0, 0) [左上角]
物件座標 (x, y)
= (280, 240)
寬:90, 高:90
向上飛行: Y 遞減 (rect.y -= speed)
向下墜落: Y 遞增 (rect.y += speed)
右下角邊界 (WIDTH, HEIGHT)
+Y 方向 (向下增加，高度 Height)

# The Heartbeat of a Game

## 1. Listen (Event Polling)

Reading OS-level hardware inputs (keyboard, mouse, window close).






## 2. Think (State Update)

Executing math and logic (calculating physics, moving coordinates, scoring).



## 3. Draw (Render)

Wiping the canvas clean and painting all objects at their their new coordinates.



Gemini Notebook

# 經典遊戲迴圈三大階段 (The Core Game Loop - 60 FPS)

## 1. 事件監聽 (Event)

- pygame.event.get()
- 讀取鍵盤按下/放開
- 滑鼠點擊與視窗關閉
- 處理單次發射等動作

## 2. 狀態更新 (Update)

- all_sprites.update()
- 計算角色位移與速度
- 子彈飛行與隕石墜落
- groupcollide 碰撞判定

## 3. 畫面繪製 (Render)

- screen.fill(BLACK)
- all_sprites.draw(screen)
- 繪製分數文字與血量
- pygame.display.flip()

循環：clock.tick(60) 限制每秒執行 60 幀，維持各電腦速度一致

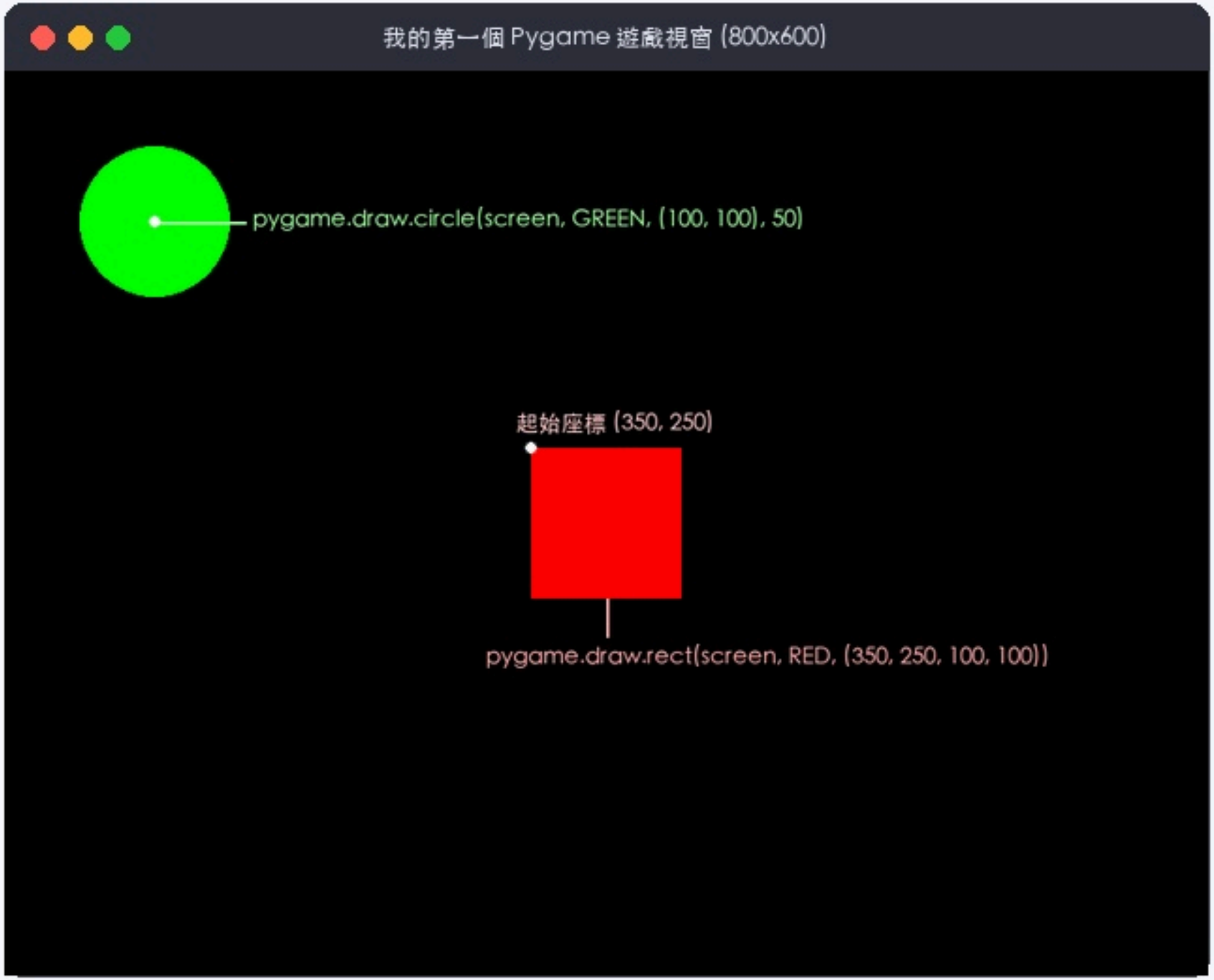
# 基礎視窗初始化與幾何繪圖

```python
import pygame, sys
pygame.init()

screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("My Pygame Window")
clock = pygame.time.Clock()

while True:
    clock.tick(60) # 限制 60 FPS
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit(); sys.exit()

    screen.fill((0, 0, 0)) # 黑色背景
    pygame.draw.rect(screen, (255, 0, 0), (350, 250, 100, 100)) # 紅色方塊
    pygame.draw.circle(screen, (0, 255, 0), (100, 100), 50)     # 綠色圓形
    pygame.display.flip() # 雙重緩衝區刷新
```


我的第一個 Pygame 遊戲視窗 (800x600)
pygame.draw.circle(screen, GREEN, (100, 100), 50)
起始座標 (350, 250)
pygame.draw.rect(screen, RED, (350, 250, 100, 100))

# Concept Check Question (CCQ 1)

**在 Pygame 遊戲設計中，關於螢幕座標系的描述，下列何者正確？**

- **A.** 原點 $(0, 0)$ 位於螢幕中心點，向上與向右為正數
- **B.** 原點 $(0, 0)$ 位於螢幕左上角，向右為 $+X$，向下為 $+Y$
- **C.** 原點 $(0, 0)$ 位於左下角，符合傳統數學幾何座標
- **D.** 當 $Y$ 座標增加時，物體會在畫面上向上移動



# CCQ 1 - 答案與解析

## 正確答案：B. 原點在左上角，向右為 $+X$，向下為 $+Y$



- **解析**：
  - 電腦顯示器硬體是由左至右、由上至下依序掃描成像的。
  - 因此 2D 視窗座標系的 $Y$ 軸正方向是「垂直向下」，增加 $Y$ 值會使物體向下移動，故選 B。

# Concept Check Question (CCQ 2)

**在遊戲主迴圈中，`clock.tick(60)` 指令的核心用途為何？**

- **A.** 限制 CPU 最大時脈以減少耗電
- **B.** 強制等待 60 毫秒後再繼續
- **C.** 限制每秒最大幀數 (FPS) 為 60，使遊戲在不同性能電腦上速度一致
- **D.** 設定遊戲倒數計時器為 60 秒



# CCQ 2 - 答案與解析

## 正確答案：C. 控制 FPS 幀率，確保跨硬體速度一致



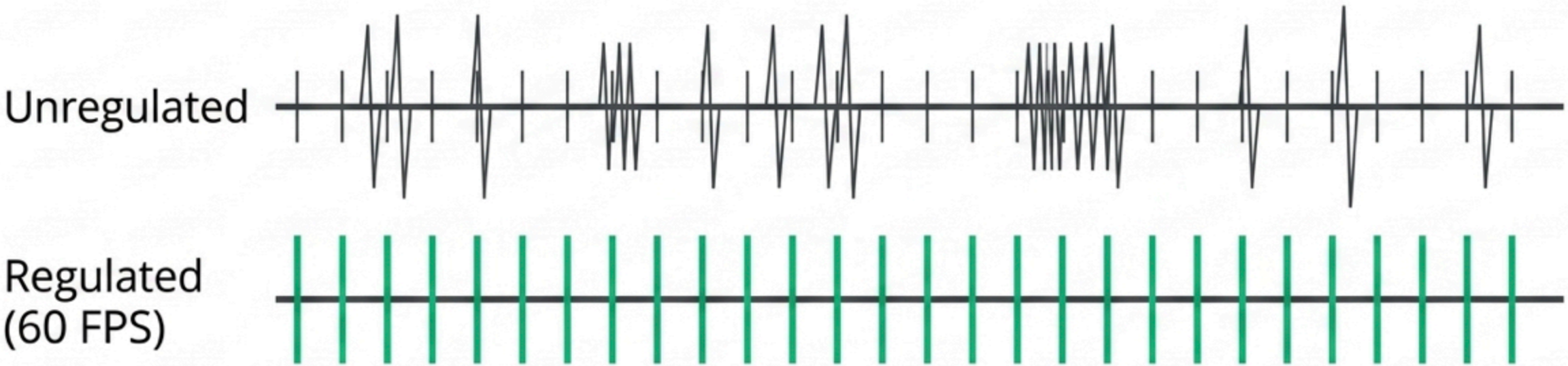
- **解析**：
  - 若未限制幀率，遊戲迴圈會在高效能 CPU 上以每秒數千次狂飆，導致物體瞬間移動且耗盡單核 CPU。
  - `clock.tick(60)` 能維持固定的時間步長，確保遊戲節奏在各平台均勻流暢，故選 C。

# 11.2 事件處理與輸入系統

- **事件佇列 (Event Queue - `pygame.event.get()`)**：
  - 紀錄單次、瞬間的離散硬體觸發（如按下空白鍵射擊、點擊滑鼠、關閉視窗）。
- **按鍵狀態輪詢 (Key State Polling - `pygame.key.get_pressed()`)**：
  - 每個幀即時檢查按鍵是否「長按壓住」，適合平滑連續運動（如左右操控飛船）。
- **滑鼠控制**：
  - `pygame.mouse.get_pos()` 取得即時游標座標。

# Time Control & Frame Rate

Without intervention, a game loop runs at maximum CPU speed. Characters would move at light-speed, varying wildly across different hardware. Using clock.tick(60) ensures consistent physics and gameplay logic across all computers.



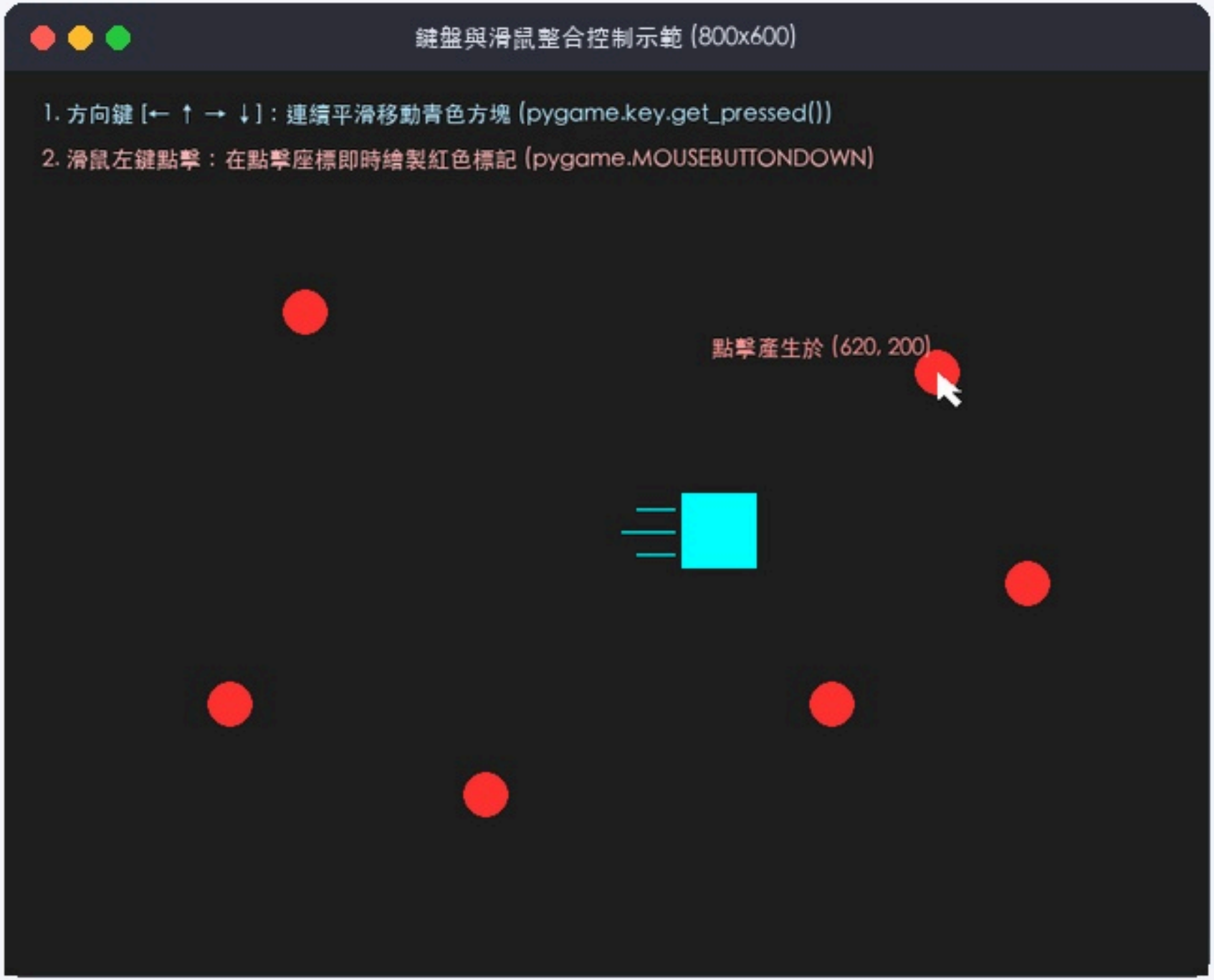

Gemini Notebook

# 鍵盤連續移動與滑鼠點擊控制

```
# 1. 處理單次事件（滑鼠左鍵點擊生成圓形）
for event in pygame.event.get():
    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        circles.append(event.pos)

# 2. 處理長按輪詢（鍵盤平滑運動）
keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT]:  block_x -= 5
if keys[pygame.K_RIGHT]: block_x += 5
if keys[pygame.K_UP]:    block_y -= 5
if keys[pygame.K_DOWN]:  block_y += 5

# 繪製畫面
screen.fill((30, 30, 30))
pygame.draw.rect(screen, (0, 255, 255), (block_x, block_y, 50, 50))
for pos in circles:
    pygame.draw.circle(screen, (255, 50, 50), pos, 15)
pygame.display.flip()
```


鍵盤與滑鼠整合控制示範 (800x600)
1. 方向鍵 [← ↑ → ↓]：連續平滑移動青色方塊 (pygame.key.get_pressed())
2. 滑鼠左鍵點擊：在點擊座標即時繪製紅色標記 (pygame.MOUSEBUTTONDOWN)
點擊產生於 (620, 200)

# Concept Check Question (CCQ 3)

**在遊戲每幀繪圖完成後呼叫 `pygame.display.flip()`，其底層「雙重緩衝區 (Double Buffering)」機制主要解決什麼問題？**

- **A.** 釋放顯示卡未使用的暫存記憶體
- **B.** 防止螢幕閃爍與撕裂，讓玩家看到完整繪製好的成品幀
- **C.** 自動計算精靈之間的碰撞
- **D.** 將 2D 向量自動渲染為 3D 視角



# CCQ 3 - 答案與解析

## 正確答案：B. 防止螢幕閃爍與撕裂



- **解析**：
  - 雙重緩衝區由「前台顯示畫布」與「後台繪製畫布」組成。
  - 所有 `draw` 操作都在後台默默進行，完成後透過 `flip()` 瞬間翻轉前台，避免玩家看到圖形逐一畫上的過程與殘影，故選 B。

# 11.3 精靈與碰撞偵測 (Sprites & Collisions)

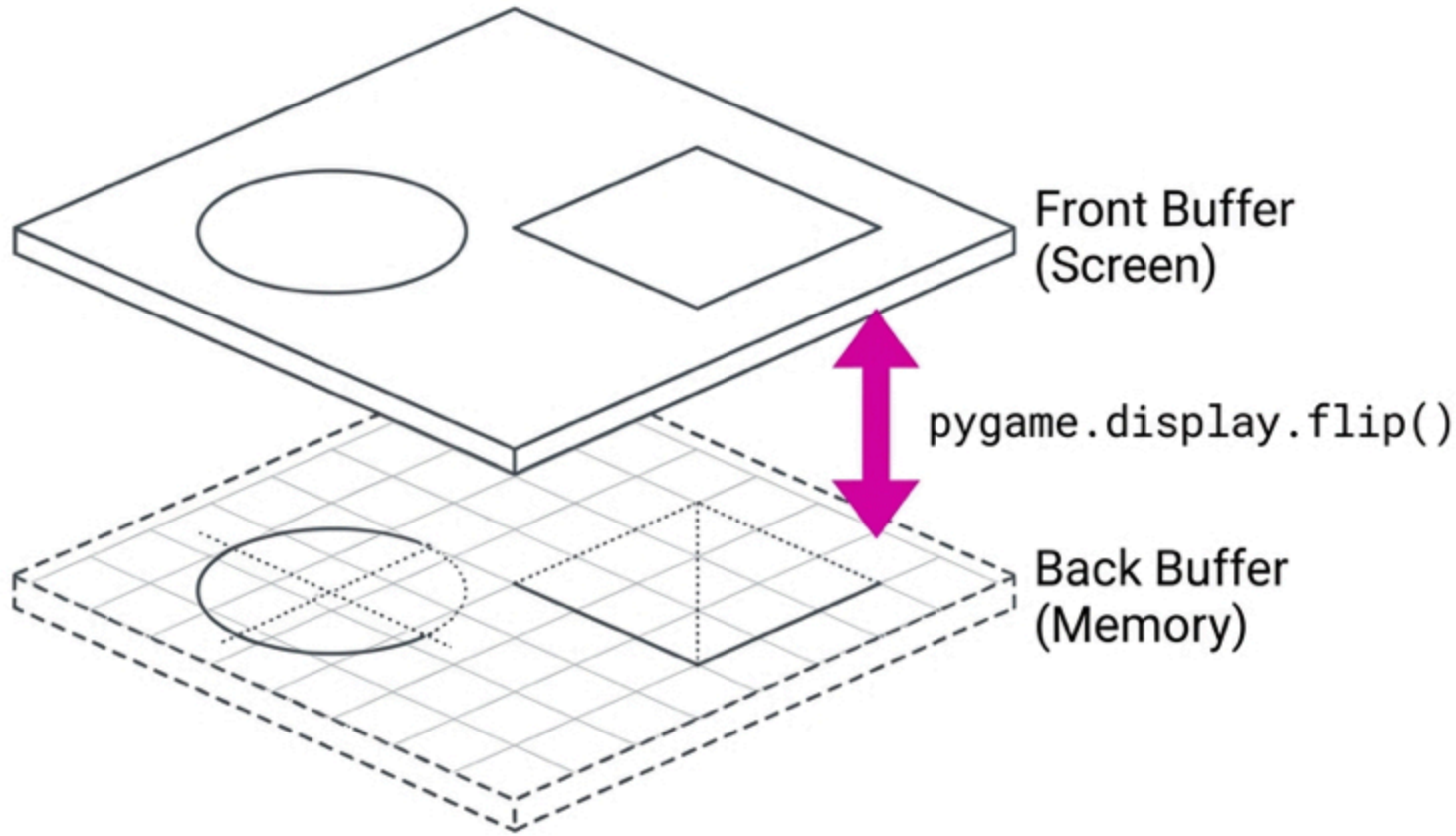
- **`pygame.sprite.Sprite` 類別**：
  - 物件導向遊戲實體基類，必須具備 `self.image` (Surface) 與 `self.rect` (Rect)。
- **`pygame.sprite.Group` 精靈群組**：
  - 一行指令 `group.update()` 批次更新所有成員。
  - 一行指令 `group.draw(screen)` 批次渲染所有實體。
- **AABB 矩形邊界碰撞偵測**：
  - `groupcollide(g1, g2, dokill1, dokill2)` 高效多對多檢測。

# The Illusion of Motion: Double Buffering

All `pygame.draw` actions happen in the hidden memory (Back Bufferffer).

Calling `pygame.display.flip()` instantly swaps the finished drawing to the screen.

Result: Prevents visible screen tearing and flicker.




Gemini Notebook

# Input Systems Architecture

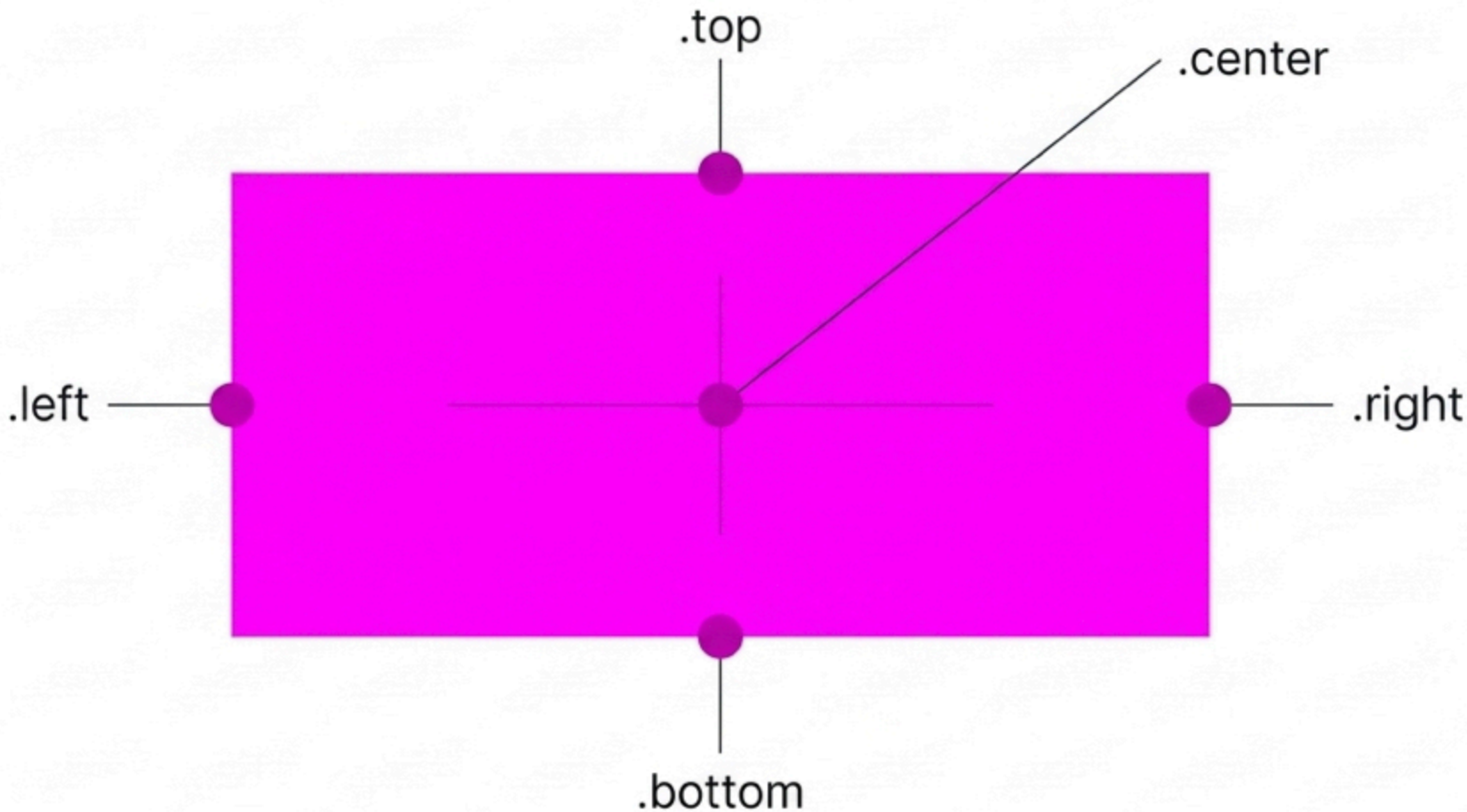
| Event Queue | vs | State Polling |
| --- | --- | --- |
| `pygame.event.get()` | | `pygame.key.get_pressed()` |
| Single, discrete pulse (key down / key up). | | Continuous boolean evaluation (True while held). |
| Firing a single bullet, opening a pause menu, closing the window. | | Smooth, continuous movement (e.g., holding arrow keys to pilot a ship). |

Gemini Notebook

# Mastering the Bounding Box

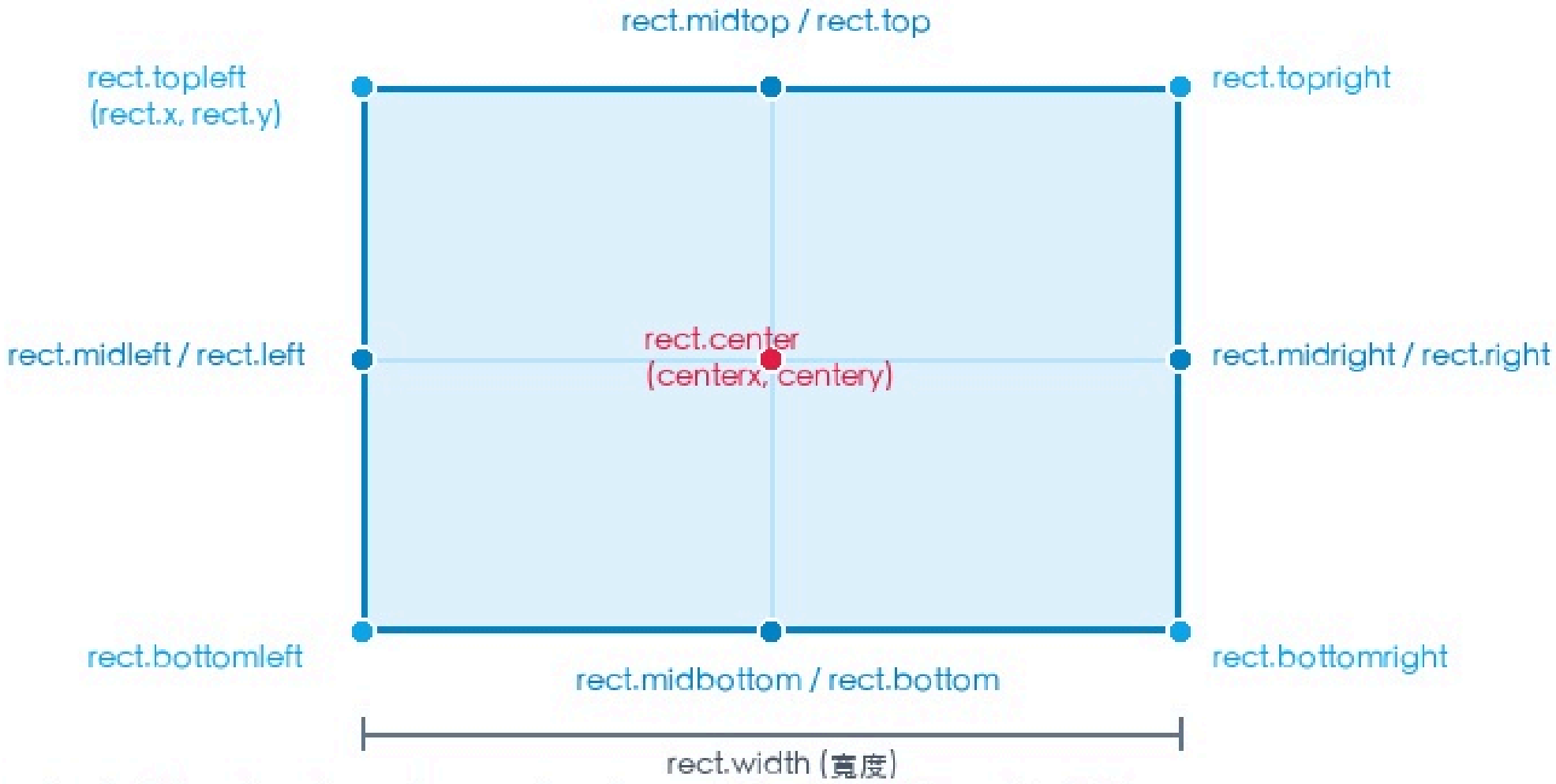
`pygame.Rect` properties are interlinked.
Modifying one dynamically updates all others.

```
# Align bullet directly to the center of the ship
bullet.rect.centerx = player.rect.centerx
bullet.rect.bottom = player.rect.top
```




Gemini Notebook

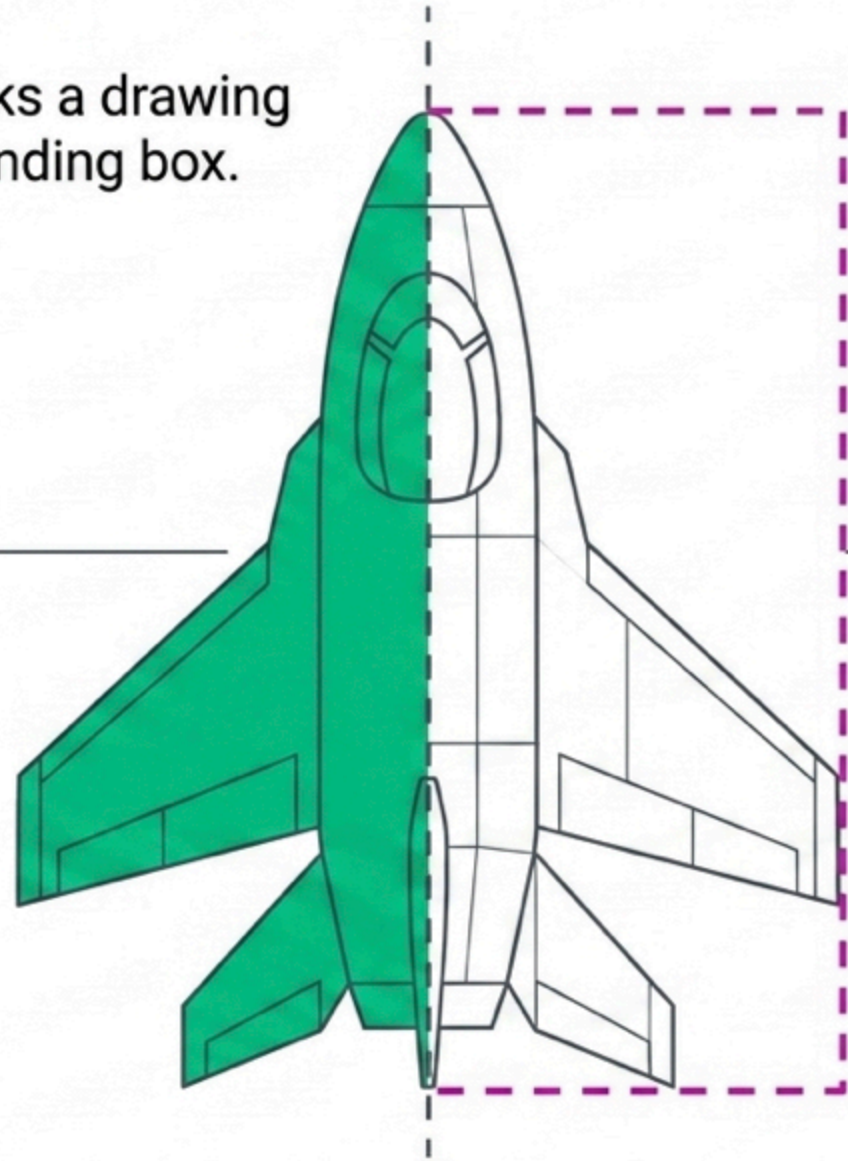
# pygame.Rect 物件內建座標定位屬性詳解




技巧：設定 bullet.rect.centerx = player.rect.centerx 可輕鬆將子彈發射位置對齊玩家中央！

# Anatomy of a Sprite

A `pygame.sprite.Sprite` links a drawing surface to a mathematical bounding box.



`self.image`
(The Canvas / Visual Appearance)

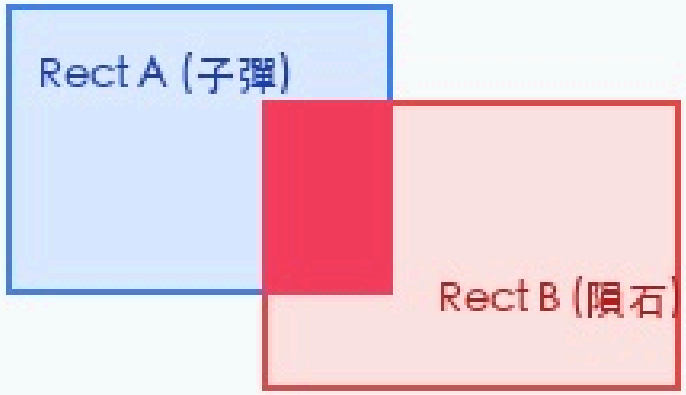
`self.rect`
(The Physics / Spatial Position)

Gemini Notebook

# Pygame 碰撞檢測機制 (Collision Detection)

## 1. 矩形邊界碰撞 (AABB / colliderect)

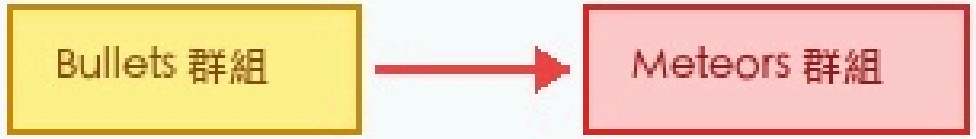
檢查兩矩形在 X 軸與 Y 軸上是否重疊




碰撞觸發重疊區 (Hits = True)

## 2. 精靈群組多對多碰撞 (groupcollide)

一行指令高效檢測兩大集合所有實體




```
pygame.sprite.groupcollide(
  bullets, meteors, True, True
)
```

命中時自動將碰撞雙方 kill() 移出畫面

# Concept Check Question (CCQ 4)

**在 Pygame 中，一個繼承自 `pygame.sprite.Sprite` 的自訂類別，初始化時必須設定哪兩個變數屬性，才能被 Sprite Group 正確管理與繪製？**

- **A.** `self.x` 與 `self.y`
- **B.** `self.image` (Surface 外觀) 與 `self.rect` (Rect 座標邊框)
- **C.** `self.speed` 與 `self.direction`
- **D.** `self.width` 與 `self.height`



# CCQ 4 - 答案與解析

## 正確答案：B. `self.image` 與 `self.rect`



- **解析**：
  - `SpriteGroup.draw()` 在遍歷成員繪圖時，會讀取每個物件的 `image` 取得畫布，並讀取 `rect` 取得繪製位置。
  - 缺少任一屬性都會引發 AttributeError，故選 B。

# Concept Check Question (CCQ 5)

**在太空射擊遊戲中，檢測「所有子彈群組」與「所有隕石群組」的碰撞並讓相撞兩者同時消滅，最佳指令為何？**

- **A.** `pygame.Rect.colliderect()`
- **B.** `pygame.sprite.spritecollide()`
- **C.** `pygame.sprite.groupcollide(bullets, meteors, True, True)`
- **D.** 撰寫雙重 for 迴圈計算每顆子彈與隕石的歐幾里得距離



# CCQ 5 - 答案與解析

## 正確答案：C.

```
groupcollide(bullets, meteors, True, True)
```

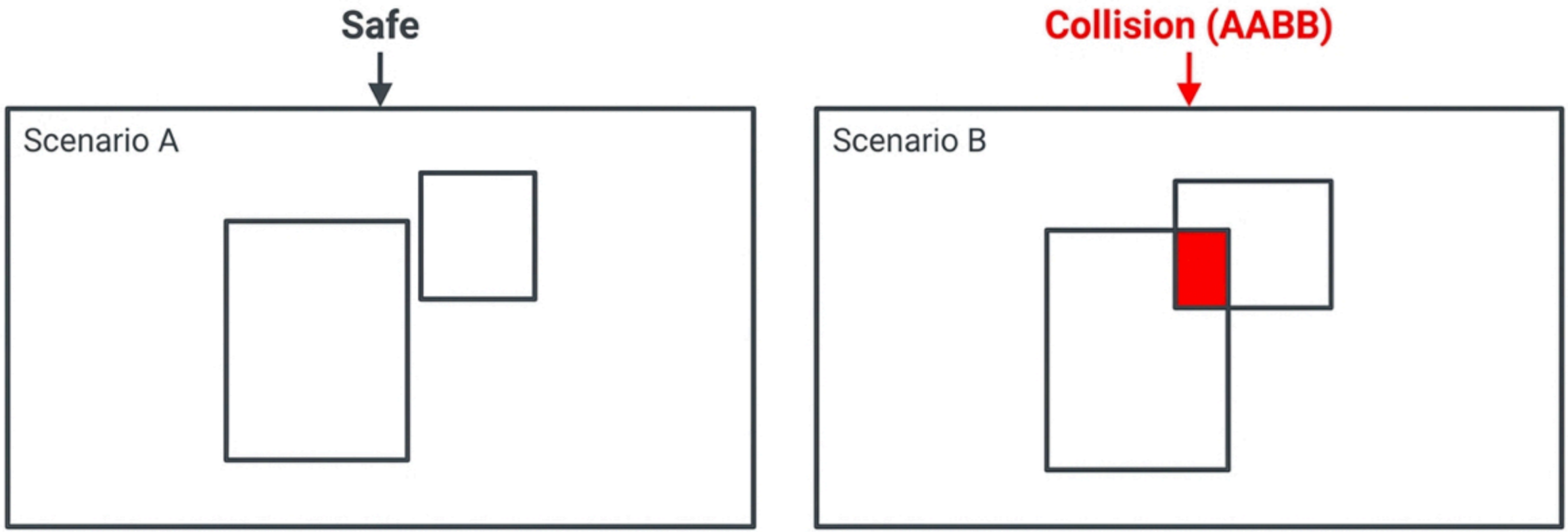


- **解析**：
  - `groupcollide` 是專為群組對群組碰撞優化的高效函式。
  - 傳入 `True, True` 可在碰撞發生時自動調用雙方成員的 `kill()` 方法將其從群組移除，一行代碼替代複雜迴圈，故選 C。

# 11.4 聲音與背景音樂整合 (Sound & Mixer)

- `pygame.mixer.music` **(背景音樂)**：
  - 適合長音訊 (mp3, ogg)。
  - **串流解碼播放**，節省記憶體，支援循環播放 (`play(-1)`)。
- `pygame.mixer.Sound` **(獨立音效)**：
  - 適合短促高頻聲音 (wav)。
  - **一次性載入記憶體**，超低延遲即時觸發 (如射擊、爆炸)。

# Physics & Collisions (AABB)




```
# The Group Solution: Efficient multi-entity collision check without nested loops
pygame.sprite.groupcollide(bullets, meteors, True, True)
```

Gemini Notebook

# 11.5 太空射擊遊戲專案開發 (Space Shooter)

- **三大核心實體設計**：
  - **Player**：受鍵盤左右控制，限制於螢幕邊界。
  - **Meteor**：由螢幕頂端隨機生成，隨機速度飄移墜落。
  - **Bullet**：按下空白鍵生成於玩家正上方，高速向上飛行。
- **遊戲生命週期與狀態機**：
  - 玩家生命值 (Lives)、計分板 (Score) 與 Game Over 重開機制。

# The Audio Engine



## `BGM (mixer.music)`

- **Keyword:** Streaming
- **Mechanism:** Decodes on the fly.
- **Use Case:** Large, long files (MP3/OGG). Saves system RAM.

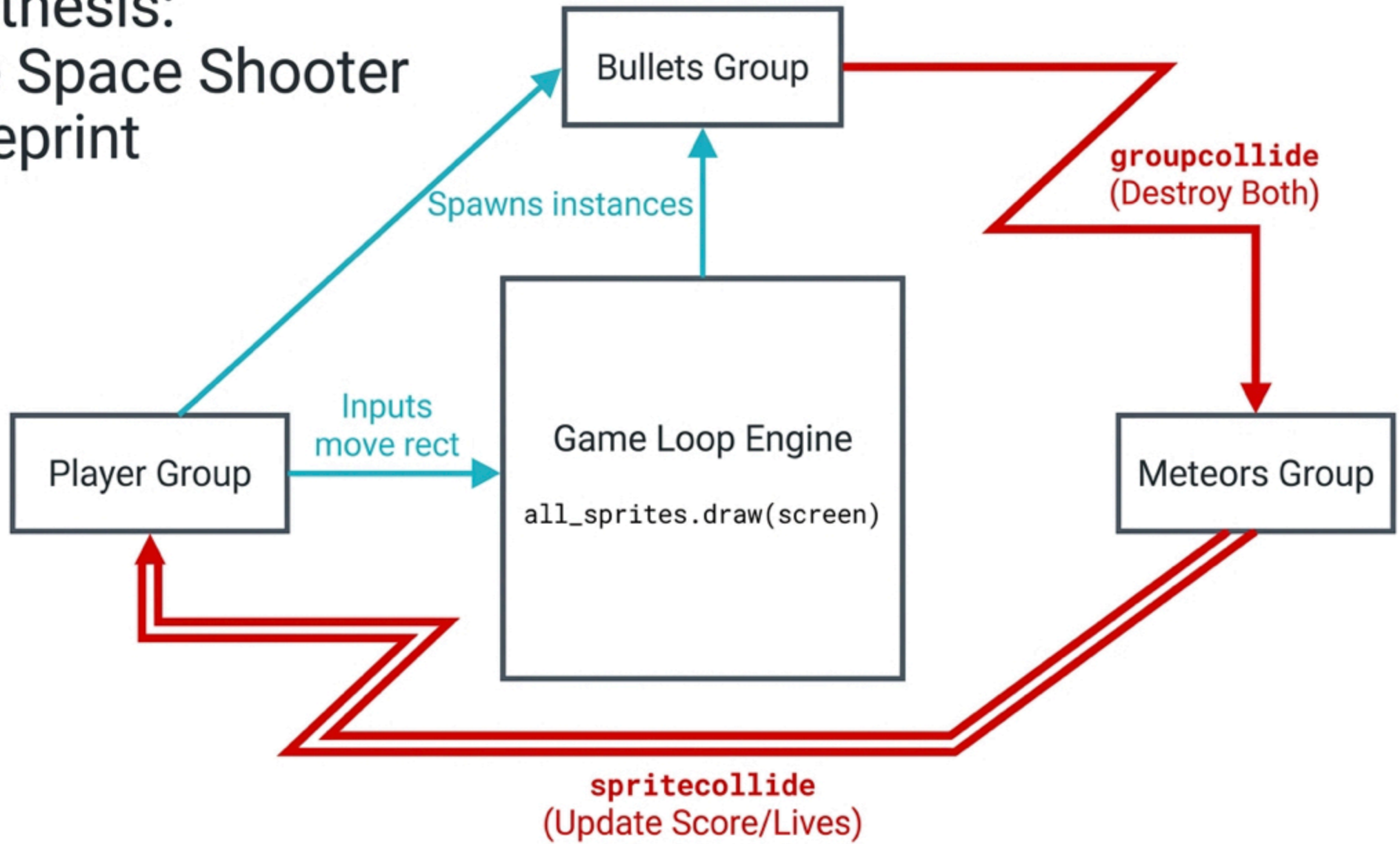


## `SFX (mixer.Sound)`

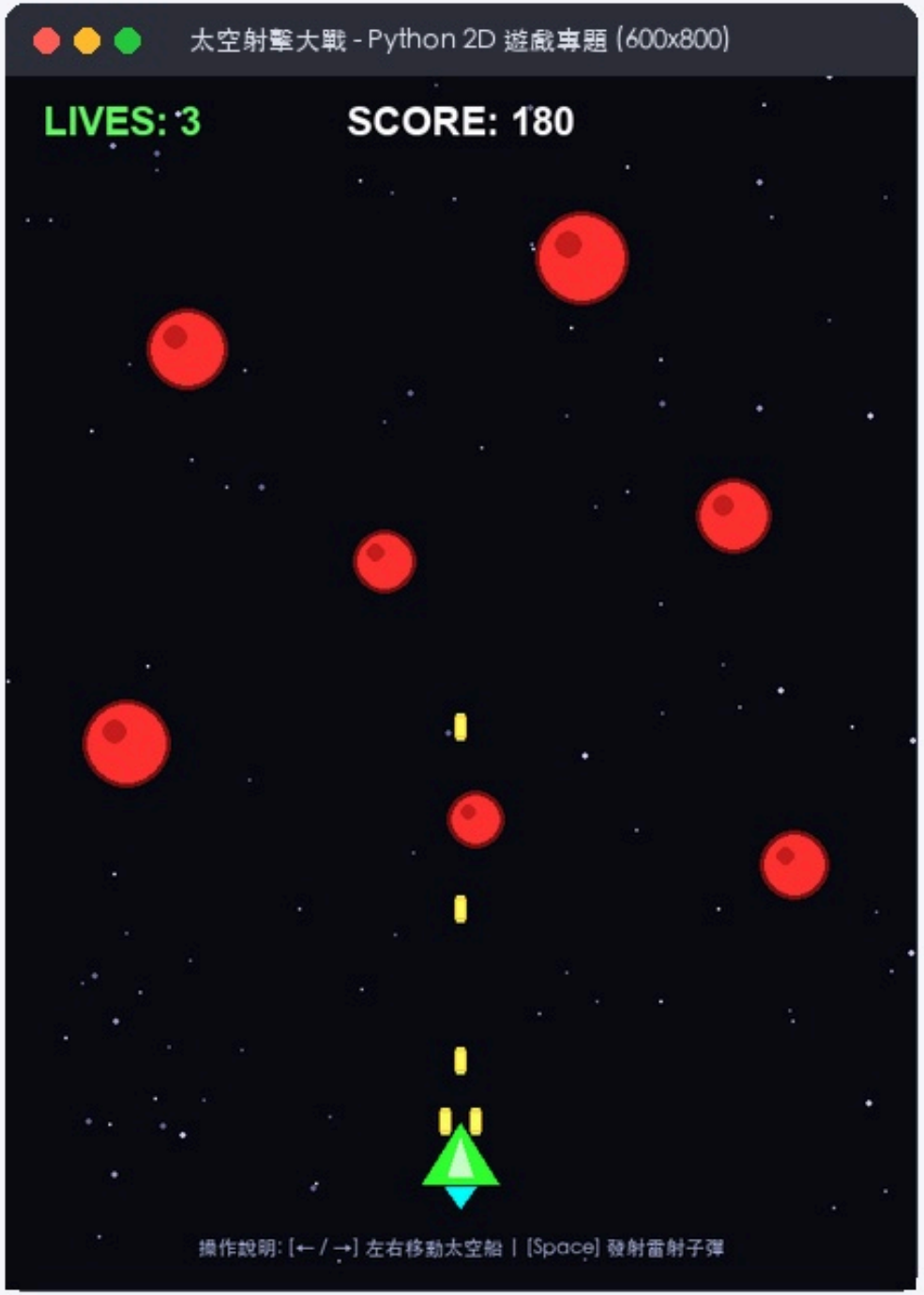
- **Keyword:** Pre-loaded
- **Mechanism:** Stored entirely in memory upon startup.
- **Use Case:** Short, high-frequency files (WAV). Guarantees zero latency for instant playback.

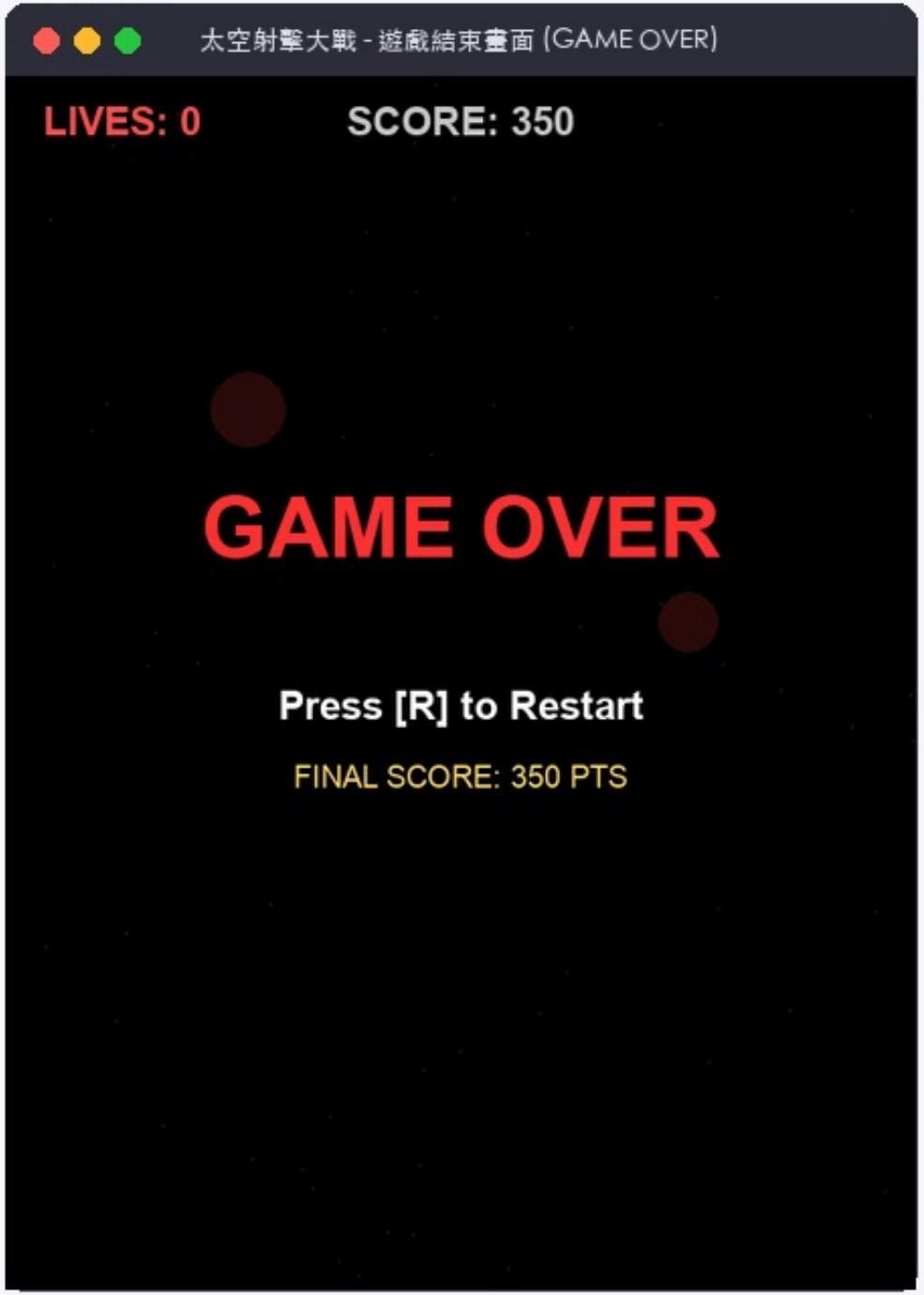
Gemini Notebook

# Synthesis: The Space Shooter Blueprint




Gemini Notebook


太空射擊大戰 - Python 2D 遊戲專題 (600x800)
LIVES: 3
SCORE: 180
操作說明: [← / →] 左右移動太空船 | [Space] 發射雷射子彈


太空射擊大戰 - 遊戲結束畫面 (GAME OVER)
LIVES: 0
SCORE: 350
GAME OVER
Press [R] to Restart
FINAL SCORE: 350 PTS

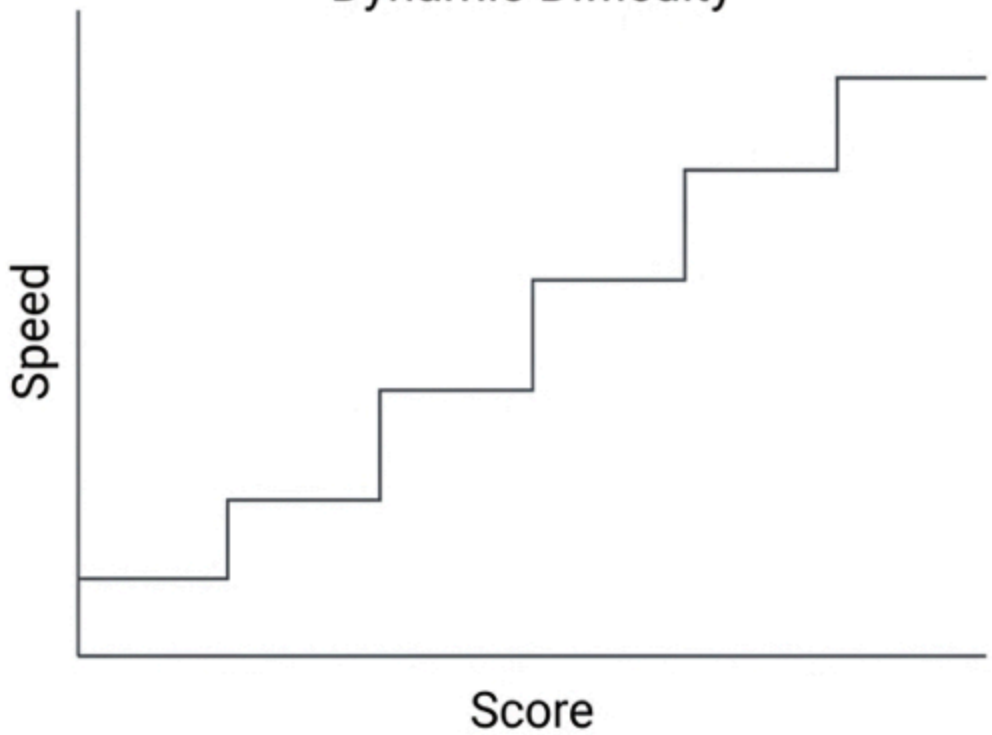
# 11.6 課後進階挑戰專題 (Particle FX)

- **粒子系統 (Particle System)**：
  - 擊中隕石時向四周迸發具備隨機初速度的小方塊粒子。
  - 粒子每幀衰減生命期 (`lifetime -= 1`)，結束時自動 `kill()`。
- **增添遊戲打擊感 (Visual Juice)**：
  - 賦予 2D 復古遊戲豐富的視覺反饋與震撼感。

# Beyond the Basics: Polish & Game Feel

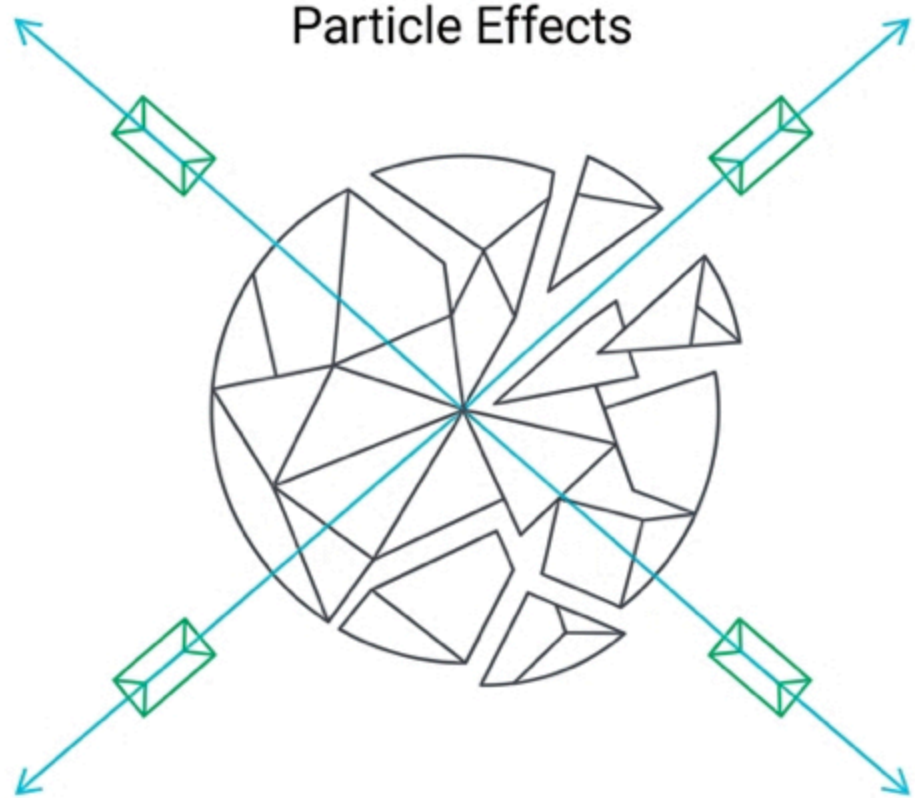
Complex mechanics are just simple objects managed within the loop.

## Dynamic Difficulty




```
speed_y += (score // 100) * 0.5
```

## Particle Effects

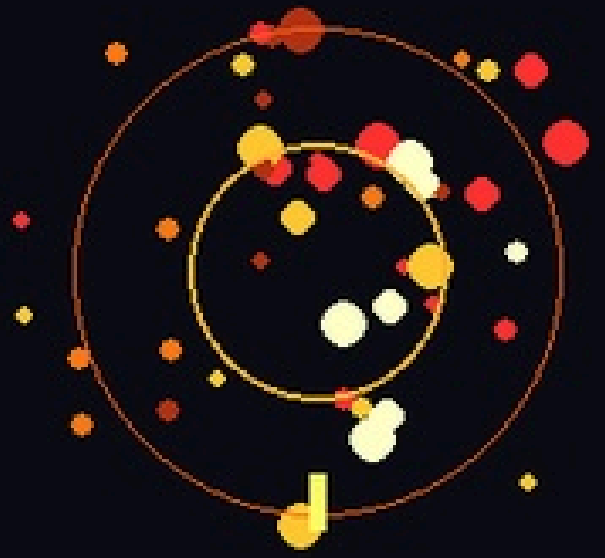


Spawning independent micro-sprites on collision with a limited frame lifetime before calling kill().

Gemini Notebook

進階挑戰專題：粒子爆炸打擊特效展示

## 擊中判定與粒子爆炸特效 (Particle Explosion Effect)



粒子隨機朝四周飄散並逐幀衰減 lifetime，大幅增強遊戲打擊感與視覺回饋